

# Cải tiến bài toán Influence Minimization via Node Blocking

Trần Văn Khánh

## 1 Bộ dữ liệu

- Sử dụng các bộ dữ liệu DBLP, EmailAll, p2p-Gnutella31, Youtube có thể được tìm thấy tại SNAP. Tìm kiếm với từ khóa SNAP + tên dataset.

- Các bộ dữ liệu được lấy về sẽ được lưu dưới dạng .txt và chỉnh sửa về form chung:

1. Dòng 1 chứa 2 số nguyên dương là số nút và số cạnh.
2. Dòng 2 chứa xâu 'directed' hoặc 'undirected' biểu thị đồ thị có hướng hay vô hướng.
3. Các dòng còn lại, mỗi dòng chứa 2 số tự nhiên  $u, v$  là 2 cạnh  $u, v$ .

- Tạo tham số xác suất lây nhiễm cạnh cho mô hình IC: xét tuân theo mô hình WC (Weighted Cascade). Cụ thể, mỗi cạnh  $(u, v)$  có trọng số xác suất  $p_{u,v} = 1/|N_{in}(v)|$ , với  $N_{in}(v)$  là tập các nút có cạnh nối tới  $v$ . Cách đặt tham số này áp dụng cho cả 2 trường hợp đồ thị có hướng và không có hướng.

- Tạo tập hạt giống  $S$ . Tìm 200 nút có bậc out-degree lớn nhất và chọn ngẫu nhiên  $|S|$  nút trong 200 nút này. Tập nút được chọn ngẫu nhiên 1 lần, lưu lại vào file kết quả seed\_node.txt và được sử dụng chung cho mọi lần mô phỏng của bộ dữ liệu đang xét (tránh trường hợp mỗi lần chạy thuật toán là một tập hạt giống khác nhau).

- Tập nút muốn chặn  $B$  cần tìm gồm có  $k = |B|$  phần tử.

## 2 Thuật toán baseline

- Thuật toán baseline để so sánh:

1. AdvancedGreedy và GreedyReplace. Link: [Minimizing the Influence of Misinformation via Vertex Blocking](#). Thuật toán này khá cơ bản nên có thể được vibe code.
2. IMin (tách riêng kết quả thu được của lower bound và upper bound với nhau thay vì chỉ hiển thị một kết quả tốt nhất đạt được). Link: [Efficient Influence Minimization via Node Blocking](#). Thuật toán này được chia sẻ mã nguồn trên github được đính kèm trong paper.

## 3 Thuật toán đề xuất

**Definition (Target Reverse Reachable Realization - TaR3):** Cho trước một nút  $v$  và nguồn  $S$ . Từ  $v$ , thực hiện duyệt đồ thị ngược về  $S$  để tìm các cạnh được kích hoạt. Nếu tồn tại đường đi từ  $S$  tới  $v$ , đồ thị TaR3 của nút  $v$ , kí hiệu là  $TaR3(v)$  là đồ thị chứa các nút và cạnh nằm trên đường đi từ  $S$  tới  $v$ . Ngược lại, đồ thị  $TaR3(v)$  là một đồ thị chứa tập nút và cạnh rỗng.

Cho trước một tập nút chặn  $B$ . Xét một  $TaR3(v)$  của một nút  $v$  cho trước. Không mất tính tổng quát, có thể thực hiện BFS để sắp xếp các nút của đồ thị  $TaR3(v)$  thành từng lớp, với lớp đầu tiên (thứ 0) chỉ có nút gốc  $S$ . Chú ý rằng nút  $v$  không nhất thiết phải nằm ở lớp cuối. Tiếp

theo, gọi  $\mathcal{P}(TaR3(v))$  là tập các đường đi từ  $S$  tới  $v$  có tính chất: i) Không có nút nào được đi qua quá một lần, ii) Chỉ được phép đi lần lượt tới một nút ở lớp ngay kế tiếp hoặc tới một nút ở cùng lớp nhưng không quá một lần. Giả sử mọi đường đi trong  $\mathcal{P}(TaR3(v))$  có vai trò bình đẳng, ta lấy mẫu một đường đi  $\rho \in \mathcal{P}(TaR3(v))$  ngẫu nhiên với phân phối đều.

Khi đó, kỳ vọng số nút cứu được khỏi  $S$  sau khi chặn tập nút  $B$  thỏa mãn:

$$D(B) \leq U(B) = \sum_{v \in V_S} \mathbb{E}_{\rho \sim \mathcal{P}(TaR3(v))} [V(\rho) \cap B \neq \emptyset]$$

Trong đó,  $V_S$  là tập các nút có thể bị ảnh hưởng bởi  $S$  trên đồ thị gốc  $G$ ,  $V(\rho)$  là tập các nút của đường đi  $\rho$ .

**Definition (Random Target Reverse Reachable Realization - RTaR3):** Cho trước một nguồn  $S$ . Chọn một nút  $v$  ngẫu nhiên, sau đó lấy mẫu đồ thị  $TaR3(v)$  ứng với  $v$ , ta thu được một đồ thị RTaR3.

**Định lý:** Hàm chặn trên của  $D(B)$  có thể được biểu diễn bởi

$$U(B) = |V_S| \cdot \mathbb{E}_{\rho \sim \mathcal{P}(RTaR3)} [V(\rho) \cap B \neq \emptyset]$$

**Algorithm 1:** LaF Path Sampling

---

**Input:** Đồ thị  $G = (V, E)$ , tập nút mầm  $S$ , nút đích  $v$ .  
**Output:** Đường đi LaF  $\rho$  phân phối đều từ một mầm bất kỳ trong  $S$  tới  $v$ , dùng tối đa 1 cạnh ngang.

```

// 1. Phân lớp BFS đa nguồn và chuẩn bị DP
1 Khởi tạo hàng đợi BFS chứa mọi nút  $s \in S$  và gán  $L(s) \leftarrow 0$ ;
2 Thực hiện BFS để gán nhãn tầng  $L(u)$  cho các nút còn lại và tìm tầng sâu nhất  $L_{\max}$ ;
3 Khởi tạo  $dp[u][0] \leftarrow 0$  và  $dp[u][1] \leftarrow 0$  cho mọi  $u \in V$ ;
4 for mỗi  $s \in S$  do
5    $dp[s][0] \leftarrow 1$ ;
// 2. Forward DP: Quét theo từng tầng (Level by Level)
6 for  $l \leftarrow 0$  to  $L_{\max}$  do
   // Xử lý các cạnh đi ngang trong cùng tầng  $l$ 
7   for mỗi cạnh  $(u, w) \in E$  sao cho  $L(u) = L(w) = l$  do
8      $dp[w][1] \leftarrow dp[w][1] + dp[u][0]$ ;
   // Lan truyền xuống tầng  $l + 1$ 
9   for mỗi cạnh  $(u, w) \in E$  sao cho  $L(u) = l$  và  $L(w) = l + 1$  do
10     $dp[w][0] \leftarrow dp[w][0] + dp[u][0]$ ;
11     $dp[w][1] \leftarrow dp[w][1] + dp[u][1]$ ;
// 3. Backward Sampling: Lấy mẫu ngược từ  $v$  về tập  $S$ 
12  $\rho \leftarrow \langle v \rangle$ ;
13  $curr \leftarrow v$ ;
   // Chọn trạng thái khởi điểm tại đích  $v$ 
14  $Z_{start} \leftarrow dp[v][0] + dp[v][1]$ ;
15 Lấy mẫu trạng thái  $state \in \{0, 1\}$  với xác suất tỷ lệ thuận với  $dp[v][state]$ ;
   // Lặp cho đến khi chạm vào bất kỳ mầm nào thuộc  $S$ 
16 while  $curr \notin S$  do
17    $Candidates \leftarrow \emptyset$ ;
   // Nếu đi từ tầng trên xuống (Cạnh tiến)
18   for mỗi  $(u, curr) \in E$  sao cho  $L(u) = L(curr) - 1$  do
19      $\leftarrow$  Thêm cặp  $(u, state)$  vào  $Candidates$  với trọng số  $W \leftarrow dp[u][state]$ ;
   // Nếu vừa nhận từ cạnh ngang (Chỉ hợp lệ nếu state đang là 1)
20   if  $state = 1$  then
21     for mỗi  $(u, curr) \in E$  sao cho  $L(u) = L(curr)$  do
22        $\leftarrow$  Thêm cặp  $(u, 0)$  vào  $Candidates$  với trọng số  $W \leftarrow dp[u][0]$ ;
   // Tính tổng trọng số và lấy mẫu bước lùi
23    $Z_{step} \leftarrow \sum_{(u, s) \in Candidates} W$ ;
24   Lấy mẫu một cặp  $(u^*, state^*) \in Candidates$  với xác suất  $\frac{W}{Z_{step}}$ ;
25   Chèn  $u^*$  vào đầu đường đi  $\rho$ ;
26    $curr \leftarrow u^*$ ;
27    $state \leftarrow state^*$ ;
28 return  $\rho$ ;

```

---

**Algorithm 2:** Thuật toán 2

---

**Input:**  $G(V, E)$ , tập seed  $S$ , ngân sách  $k$ , tham số xấp xỉ  $(\beta, \delta)$ , sai số  $\epsilon$ .  
**Output:** Tập nút cần chặn  $B_U$ .

- 1  $ON \leftarrow$  tập các nút được kích hoạt trực tiếp bởi  $S$ ;
- 2 **if**  $|ON| \leq k$  **then**
- 3     **return**  $B_U \leftarrow ON$ ;
- 4  $OPT^L \leftarrow$  hàm chặn dưới của  $U(B_U^o)$ ;
- 5  $\theta_{\max} \leftarrow \frac{2|V_S| \left( (1-1/e) \sqrt{\ln \frac{6}{\delta}} + \sqrt{(1-1/e) \left( \ln \left( \frac{|V_S| - |S|}{k} \right) + \ln \frac{6}{\delta} \right)} \right)}{\epsilon^2 OPT^L}$ ;
- 6  $\theta_0 \leftarrow \theta_{\max} \cdot \epsilon^2 OPT^L / |V_S|$ ;
- 7  $i_{\max} \leftarrow \lceil \log_2 \frac{\theta_{\max}}{\theta_0} \rceil$ ;
- 8 Sinh 2 dãy đường đi  $\mathbb{P}_1, \mathbb{P}_2$  gồm  $\theta_0$  phần tử theo thuật toán 1;
- 9  $a_1 \leftarrow \ln \frac{3i_{\max}}{\delta}$ ,  $a_2 \leftarrow \ln \frac{3i_{\max}}{\delta}$ ;
- 10 **for**  $i \leftarrow 1$  **to**  $i_{\max}$  **do**
- 11      $B_U \leftarrow \text{Max-Coverage}(\mathbb{P}_1, k)$ ;  
       // Cập nhật cận dưới
- 12      $\sigma^L(B_U) \leftarrow \left( \left( \sqrt{\text{Cov}_{\mathbb{L}_2}(B_U) + \frac{2a_1}{9}} - \sqrt{\frac{a_1}{2}} \right)^2 - \frac{a_1}{18} \right) \cdot \frac{|V_S|}{|\mathbb{P}_2|}$ ;  
       // Tính Covarage cận trên
- 13      $\text{Cov}_{\mathbb{P}_1}^u(B_U^o) \leftarrow \min_{0 \leq j \leq k} \left( \text{Cov}_{\mathbb{P}_1}(B_j) + \sum_{v \in \max MC(B_j, k)} \text{Cov}_{\mathbb{P}_1}(v \mid B_j) \right)$ ;  
       // Cập nhật cận trên
- 14      $\sigma^U(B_U^o) \leftarrow \left( \sqrt{\text{Cov}_{\mathbb{P}_1}^u(B_U^o) + \frac{a_2}{2}} + \sqrt{\frac{a_2}{2}} \right)^2 \cdot \frac{|V_S|}{|\mathbb{P}_1|}$ ;
- 15     **if**  $\sigma^L(B_U) / \sigma^U(B_U^o) \geq 1 - 1/e - \epsilon$  **or**  $i = i_{\max}$  **then**
- 16         **return**  $B_U$ ;
- 17     Nhân đôi kích thước của  $\mathbb{P}_1$  và  $\mathbb{P}_2$  bằng các dãy đường đi mới;

---

## 4 Thực nghiệm

Giá trị mặc định của  $|S|$  là 10,  $k$  là 100.

- So sánh các thuật toán với nhau về số nút cứu được khi  $k$  nhận giá trị là 100, 200, 300, 400, 500.

- So sánh các thuật toán với nhau về thời gian chạy (đơn vị là giây, thời gian chạy tối đa là 1e5 s) khi  $k$  nhận giá trị là 100, 200, 300, 400, 500.

- So sánh các thuật toán với nhau về số nút cứu được khi  $|S|$  nhận giá trị là 10, 20, 30, 40, 50.

- So sánh các thuật toán với nhau về thời gian chạy (đơn vị là giây, thời gian chạy tối đa là 1e5 s) khi  $|S|$  nhận giá trị là 10, 20, 30, 40, 50.

- So sánh tỷ lệ xấp xỉ của thuật toán SandIMin-upper bound ( $D_U(.)$ ) và thuật toán đề xuất ( $U(.)$ ). Thuật toán SandIMin-upper bound có tỷ lệ xấp xỉ là  $\frac{D(B_U)}{D_U(B_U)}(1 - 1/e - \epsilon)$ , tương tự thuật toán đề xuất có tỷ lệ xấp xỉ là  $\frac{D(B_U)}{U(B_U)}(1 - 1/e - \epsilon)$  ( $B_U$  là phương án trả về bởi thuật toán 2).